



**UNIVERSIDAD
DE GRANADA**



Práctica 1 - Desarrollo de Software

Claudio Rivas Boza
Aitor Fernández Pérez
Gerardo Pérez Triviño

March 22, 2026

1 Patrón Factoría Abstracta en Java

1.1 Patrón de diseño utilizado

Para realizar el siguiente ejercicio hemos usado el patrón "Abstract Factory" (Factoría Abstrata). En concreto, para crear los objetos de nuestra factoría abstracta usaremos el patrón método factoría, aunque también se podría haber usado el patrón prototipo.

Según la teoría, el patrón **Factoría Abstracta** proporciona una interfaz para crear **familias de objetos relacionados o dependientes sin especificar sus clases concretas**. Este patrón es especialmente útil cuando un sistema debe ser independiente de cómo se crean, componen y representan sus productos.

En nuestro caso existen dos familias de productos claramente diferenciadas:

- **Familia Competitiva:** formada por `PartidaCompetitiva` y `JugadorCompetitivo`.
- **Familia Casual:** formada por `PartidaCasual` y `JugadorCasual`.

La **ventaja principal** de aplicar este patrón es que separa la creación de objetos de su uso: el código cliente (`Main`) trabaja siempre con los tipos abstractos `Partida`, `Jugador` y `FactoriaPartidaYJugador`, sin conocer nunca qué clase concreta se está instanciando.

1.2 Diagrama de clases UML

El diagrama de clases UML será el siguiente:

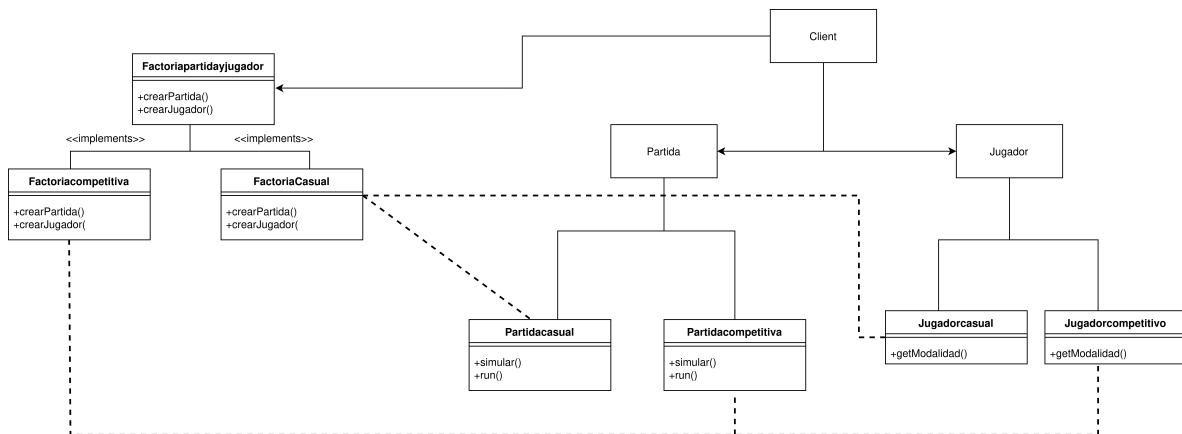


Figure 1: Diagrama de clases UML del patrón Factoría Abstracta.

1.3 Estructura de clases e interfaces

1.3.1 Interfaz FactoriaPartidaYJugador

Es la **factoría abstracta** del diseño. Define el contrato que deben cumplir todas las factorías concretas, declarando los dos métodos de fabricación públicos exigidos por el enunciado:

Se define como **interface** ya que, tal y como se indica en la teoría, una interfaz en Java "define un contrato que debe ser implementado por las clases, sin proporcionar implementación", lo cual se ajusta a nuestro caso al necesitar únicamente declarar los métodos de fabricación sin compartir lógica común.

1.3.2 Clases abstractas Partida y Jugador

Son los **productos abstractos** del patrón. Se definen como clases abstractas porque, a diferencia de una interfaz, permiten compartir código base entre las subclases: `Partida` comparte el atributo `ArrayList<Jugador>` y la duración de 60 segundos, y `Jugador` comparte el identificador `id`.

`Partida` implementa además `Runnable` para poder ser ejecutada como hebra, tal como recomienda el enunciado:

```
J Partida.java > Language Support for Java(TM) by Red Hat > Partida > run()
1 import java.util.ArrayList;
2 public abstract class Partida implements Runnable{ //Al utilizar Runnable hay que utilizar
3     protected ArrayList<Jugador> jugadores;
4     protected int duracion=60;
5     public Partida(ArrayList<Jugador> jugadores){
6         this.jugadores = jugadores;
7     }
8     public abstract void simular(); //porcentaje de abandono
9
10    public abstract void run(); //este es el método obligatorio al implementar Runnable
11 }
```

Figure 2: Clase abstracta `Partida`: producto abstracto que implementa `Runnable`.

1.3.3 Factorías

Implementan la interfaz `FactoriaPartidaYJugador` y son las responsables de instanciar cada familia de productos. Aquí es donde actúa el **Método Factoría**: cada método delega la creación en la clase concreta correspondiente a su modalidad.

1.3.4 Partida competitiva y casual

Son las implementaciones específicas de cada familia. `PartidaCompetitiva` define la lógica del 20% de abandonos, y `PartidaCasual` la del 10%. Los abandonos se producen todos al mismo tiempo, en la mitad de la partida:

```
J Partidacompetitiva.java > ...
1 import java.util.ArrayList;
2
3 public class Partidacompetitiva extends Partida {
4
5     public Partidacompetitiva(ArrayList<Jugador> jugadores) {
6         super(jugadores);
7     }
8
9     @Override
10    public void simular() {
11        int abandono = (int) (jugadores.size() * 0.1);
12        for (int i = 0; i < abandono; i++) {
13            int index = (int) (Math.random() * jugadores.size()); //Para eliminar un jugador aleatorio
14            Jugador j = jugadores.remove(index);
15            System.out.println("Jugador " + index + " ha abandonado la partida competitiva.");
16        }
17    }
18
19    @Override
20    public void run() {
21        System.out.println("Partida Casual iniciada con " + jugadores.size() + " jugadores.");
22        try {
23            Thread.sleep(duracion / 2 * 1000); // mitad de la partida
24            simular(); // todos abandonan a la vez
25            Thread.sleep(duracion / 2 * 1000); // resto de la partida
26        } catch (InterruptedException e) {
27            e.printStackTrace();
28        }
29        System.out.println("Partida Casual finalizada. Jugadores restantes: " + jugadores.size());
30    }
31 }
```

Figure 3: `PartidaCompetitiva`: implementación con 20% de abandonos simultáneos.

1.4 Cambios respecto al diseño original

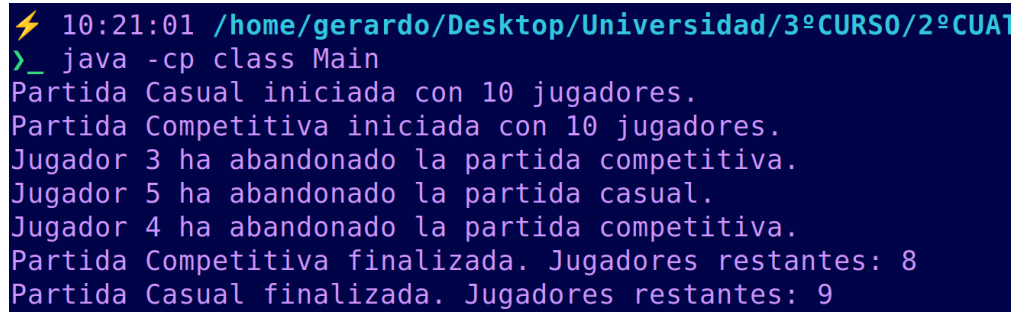
Aunque el enunciado indica que `Partida` y `Jugador` pueden definirse como clases abstractas o interfaces, se ha optado por **clases abstractas** en ambos casos. Esto se debe a que, tal y como se explica en la teoría, una clase abstracta *“permite compartir código base y puede tener métodos con implementación”*, mientras que una interfaz no puede tener atributos ni constructores.

En nuestro caso, **Partida** necesita compartir entre sus subclases el atributo `ArrayList<Jugador>` y la duración de 60 segundos, y **Jugador** necesita compartir el identificador `id`. Si se hubieran definido como interfaces, cada subclase tendría que redefinir estos atributos por separado, introduciendo código redundante, por ello se definieron como abstractas.

1.5 Ejemplos de ejecución

Para compilar usaremos `javac *.java` y para ejecutar al tener los `.java` y los `.class` en clases distintas usaremos el comando: `java -cp class Main`

Un ejemplo del resultado de su ejecución es el siguiente:



```
⚡ 10:21:01 /home/gerardo/Desktop/Universidad/3ºCURSO/2ºCUATRIEN  
>_ java -cp class Main  
Partida Casual iniciada con 10 jugadores.  
Partida Competitiva iniciada con 10 jugadores.  
Jugador 3 ha abandonado la partida competitiva.  
Jugador 5 ha abandonado la partida casual.  
Jugador 4 ha abandonado la partida competitiva.  
Partida Competitiva finalizada. Jugadores restantes: 8  
Partida Casual finalizada. Jugadores restantes: 9
```

Figure 4: Simulación de ejecución

2 Patrón Decorator en Python

2.1 Introducción

El objetivo de este ejercicio es desarrollar un programa en Python que integre la API de Hugging Face para procesar texto mediante modelos de lenguaje (*LLMs*), aplicando el patrón de diseño **Decorator** para extender su funcionalidad de forma modular.

El sistema parte de un modelo de resumen base y permite añadir dinámicamente dos capacidades adicionales: la traducción del resumen al español y el análisis de sentimientos del texto resultante. Toda la configuración del programa —modelos empleados y credenciales de acceso— se externaliza en un archivo `config.json` y `.env`, añadiéndolo al `.gitignore` para garantizar que Git no lo rastree, evitando valores *hardcodeados* y facilitando su mantenimiento.

2.2 Instalación y dependencias

Las dependencias del proyecto son las siguientes:

- **requests**: llamadas HTTP a la API de Hugging Face.
- **python-dotenv**: carga del token desde el archivo `.env`.

La instalación se realiza con:

```
sudo apt install python3-dotenv python3-requests
```

No se requiere instalar ningún modelo localmente: todas las inferencias se delegan a la *Inference API* de Hugging Face mediante llamadas HTTP, lo que elimina la necesidad de hardware especializado y permite cambiar de modelo simplemente modificando `config.json`.

2.3 Configuración del proyecto

2.3.1 El archivo `.env` y la gestión de credenciales

El token de Hugging Face es una credencial personal que autentica las llamadas a la API. Incluirlo directamente en `config.json` supondría un riesgo de seguridad, ya que cualquier persona con acceso al repositorio —incluso a través del historial de Git— podría utilizarlo de manera no autorizada.

La solución estándar es separar configuración y secretos: `config.json` contiene los parámetros no sensibles y se incluye en el repositorio, mientras que `.env` almacena únicamente el token y se excluye mediante `.gitignore`. La librería `python-dotenv` se encarga de cargar automáticamente las variables del archivo `.env` en el entorno del proceso.

2.3.2 Estructura del `config.json`

El archivo `config.json` centraliza todos los parámetros configurables del sistema:

```
{
  "texto": "Peter Parker is a fictional superhero appearing...",
  "model_llm": "facebook/bart-large-cnn",
  "model_translation": "Helsinki-NLP/opus-mt-en-es",
  "model_sentiment": "nlptown/bert-base-multilingual-uncased-sentiment"
}
```

2.4 Diagrama UML

La figura 5 muestra el diagrama de clases del sistema, donde se aprecia la jerarquía del patrón Decorator: la clase abstracta `LLM` actúa como componente, `BasicLLM` como implementación concreta, y los decoradores (`LLMDecorator`, `TranslationDecorator`, `SentimentDecorator`) extienden la funcionalidad sin modificar las clases base.

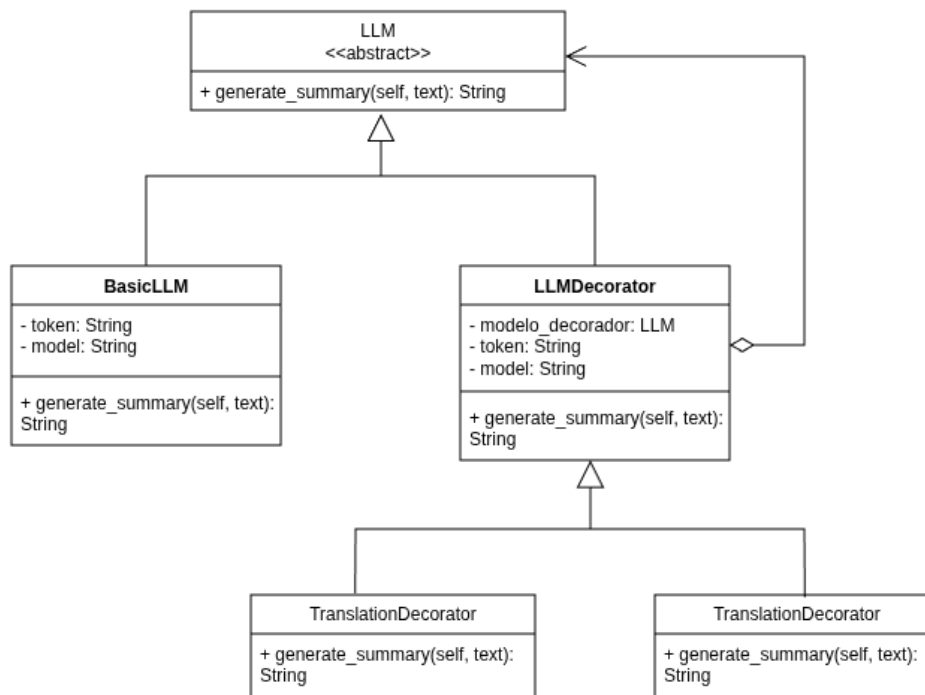


Figure 5: Diagrama de clases — Patrón Decorator

2.5 Implementación

2.5.1 Interfaz LLM y clase BasicLLM

La clase abstracta **LLM** define el contrato que deben cumplir tanto el componente base como todos los decoradores a través del método `generate_summary(text: str) -> str`. Esta uniformidad es la que hace posible que los decoradores sean intercambiables y apilables.

BasicLLM implementa esta interfaz realizando una llamada a la *Inference API* de Hugging Face con el modelo de resumen configurado. Recibe el modelo y el token en su constructor, manteniendo la lógica de red desacoplada de cualquier capa superior.

2.5.2 Clase base LLMDecorator

Para evitar repetir la lógica de delegación en cada decorador concreto, se introduce la clase intermedia **LLMDecorator**, que hereda de **LLM** y mantiene una referencia al componente decorado. Los decoradores concretos heredan de esta clase y únicamente sobrescriben `generate_summary` con su funcionalidad adicional.

2.5.3 Decoradores concretos

TranslationDecorator invoca al componente decorado para obtener el resumen y lo envía al modelo de traducción, devolviendo el texto resultante en español.

SentimentDecorator obtiene el texto del componente decorado y lo pasa al modelo de clasificación de sentimientos, añadiendo al final la etiqueta resultante entre corchetes.

2.5.4 Composición en el cliente

El código cliente instancia el **BasicLLM** y lo envuelve progresivamente en los decoradores, generando cuatro resultados: resumen básico, resumen traducido, resumen con sentimiento, y una combinación de los tres. En esta última, **TranslationDecorator** se aplica antes que **SentimentDecorator** para que el análisis opere sobre el texto ya en español, que es el idioma que maneja el sistema europeo.

3 Patrón Strategy en python

3.1 Importación de los paquetes

Lo primero que vamos a hacer es importar todos los paquetes necesarios para poder usar BeautifulSoup y Selenium, además de csv para la escritura en archivos.

```
import requests
import csv
from abc import ABC, abstractmethod
from bs4 import BeautifulSoup
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

Las últimas cuatro líneas nos sirven para configurar el selenium indicando que usaremos Chrome como navegador, para buscar elementos en la página usaremos By, para esperar a que la web cargue o se actualice usamos WebDriverWait y para definir condiciones durante la espera usaremos EC.

3.2 Configuración básica y función para la búsqueda de los elementos con BeautifulSoup

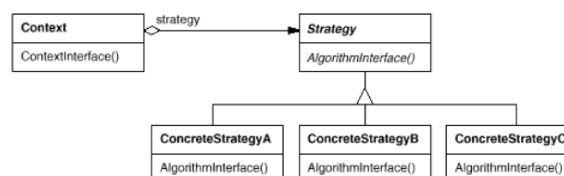
Para hacer más fácil el código y la búsqueda de cada elemento, crearemos un array con todos los elementos a buscar y una función para encontrar cada uno de ellos dentro del propio html

```
URL = "https://www.scrapethissite.com/pages/forms/?page_num=1"
CAMPOS = ["Team Name", "Year", "Wins", "Losses", "OT Losses", "Win %", "Goals For", "Goals Against", "+/-"]

def getURLs(soup):
    datos = []
    filas = soup.select("tr.team")
    for fila in filas:
        columnas = fila.find_all('td')
        if len(columnas) >= 9:
            datos.append({
                "Team Name": columnas[0].text.strip(),
                "Year": columnas[1].text.strip(),
                "Wins": columnas[2].text.strip(),
                "Losses": columnas[3].text.strip(),
                "OT Losses": columnas[4].text.strip(),
                "Win %": columnas[5].text.strip(),
                "Goals For": columnas[6].text.strip(),
                "Goals Against": columnas[7].text.strip(),
                "+/-": columnas[8].text.strip()
            })
    return datos
```

3.3 Uso del patrón Strategy

Como tenemos en las diapositivas, el diagrama UML del patrón estrategia es el siguiente:



Con este diagrama podemos orientarnos en la creación del patrón para nuestro caso particular. Para ello, crearemos cuatro clases:

- Clase Estrategia (con el método abstracto ejecutar)
- Clase Estrategia_BeautifulSoup

Con la función ejecutar propia de este método de web crawling

- Clase Estrategia_Selenium

Con la función ejecutar propia de este método de web crawling

- Clase Contexto (con el inicializador y la función ejecutar_estrategia)

```
class Estrategia(ABC):
    @abstractmethod
    def ejecutar(self) -> list:
        pass

class Estrategia_BeautifulSoup(Estrategia):
    def ejecutar(self) -> list:
        datos = []

        # Iterar para las 5 primeras páginas
        for pagina in range(1, 6):
            response = requests.get(URL.format(pagina))
            soup = BeautifulSoup(response.text, 'html.parser')
            datos.extend(getURLs(soup))

        return datos

class Estrategia_Selenium(Estrategia):
    def ejecutar(self) -> list:
        opciones = webdriver.ChromeOptions()
        opciones.add_argument("--headless")
        driver = webdriver.Chrome(service=Service(), options=opciones)

        datos = []

        # Iterar para las 5 primeras páginas
        for pagina in range(1, 6):
            driver.get(URL.format(pagina))

            WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.CSS_SELECTOR, "tr.team")))

            filas = driver.find_elements(By.CSS_SELECTOR, "tr.team")
            for fila in filas:
                columnas = fila.find_elements(By.TAG_NAME, 'td')
                if len(columnas) >= 9:
                    datos.append({
                        "Team Name": columnas[0].text.strip(),
                        "Year": columnas[1].text.strip(),
                        "Wins": columnas[2].text.strip(),
                        "Losses": columnas[3].text.strip(),
                        "OT Losses": columnas[4].text.strip(),
                        "Win %": columnas[5].text.strip(),
                        "Goals For": columnas[6].text.strip(),
                        "Goals Against": columnas[7].text.strip(),
                        "+/-": columnas[8].text.strip()
                    })
            driver.quit()

        return datos
```

```
class Contexto:
    def __init__(self, estrategia: Estrategia):
        self.estrategia = estrategia

    def ejecutar_estrategia(self) -> list:
        return self.estrategia.ejecutar()
```

3.4 Función para el guardado de los datos

Para el guardado de los datos en un archivo csv hemos creado la función propia "guardar_csv". En dicha función nos encargamos de crear el archivo y de añadir tanto las cabeceras con los datos buscados como los datos encontrados

```
def guardar_csv(datos, nombre_archivo):
    with open(nombre_archivo, mode='w', newline='', encoding='utf-8') as archivo_csv:
        writer = csv.DictWriter(archivo_csv, fieldnames=CAMPOS)
        writer.writeheader()
        writer.writerows(datos)
```


3.5 Función Main

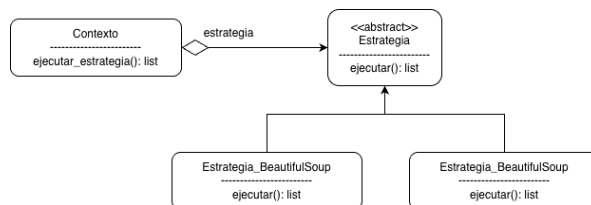
Para finalizar con el código tenemos el main, en dicho main le preguntamos en tiempo de ejecución al usuario qué tipo de programa quiere usar para el web crawling y una vez se guarden los datos en el fichero nuevo creado se le notifica.

```
if __name__ == "__main__":
    print("Selecciona una estrategia:")
    print("1. BeautifulSoup")
    print("2. Selenium")
    opcion = input("Ingrese el número de la estrategia que desea usar: ")

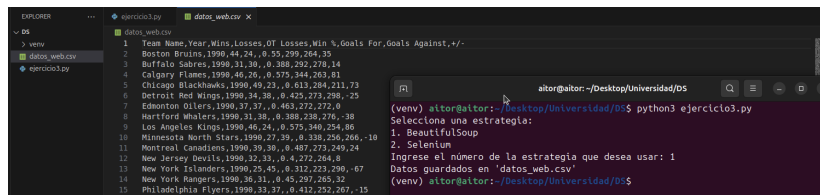
    if opcion == "1":
        estrategia = Estrategia_BeautifulSoup()
    elif opcion == "2":
        estrategia = Estrategia_Selenium()
    else:
        print("Opción no válida")
        exit()

    contexto = Contexto(estrategia)
    datos = contexto.ejecutar_estrategia()
    guardar_csv(datos, "datos_web.csv")
    print("Datos guardados en 'datos_web.csv'")
```

3.6 Diagrama UML



3.7 Resultado final obtenido



3.8 Enlaces usados para el conocimiento del tema

- <https://medium.com/@igorzabukovec/automate-web-crawling-with-selenium-python-part-1-85113660de9>
- <https://medium.com/geekculture/part-1-crawling-a-website-using-beautifulsoup-and-requests-413ff>
- <https://selenium-python.readthedocs.io/getting-started.html#simple-usage>

4 Patrón de Filtros de Intercepción en Ruby

4.1 Descripción del problema

El ejercicio consiste en construir un sistema de autenticación que valide los credenciales de un usuario, su correo y contraseña, antes de dejarlo pasar para comprobar la validez de ambos.

Nuestro sistema se encargará de:

- Pedir al usuario que escriba su correo y su contraseña.
- Comprobar el correo con dos reglas independientes: que haya texto antes del @ y que el dominio sea `gmail.com` o `hotmail.com`. Como extra he añadido que también se puedan introducir correos del tipo `correo.ugr.es` y `go.ugr.es`
- Comprobar la validez de la contraseña con tres reglas independientes: longitud mínima, que contenga al menos un número y que contenga al menos un carácter especial.
- Cada regla se encuentra en su propia clase correspondiente

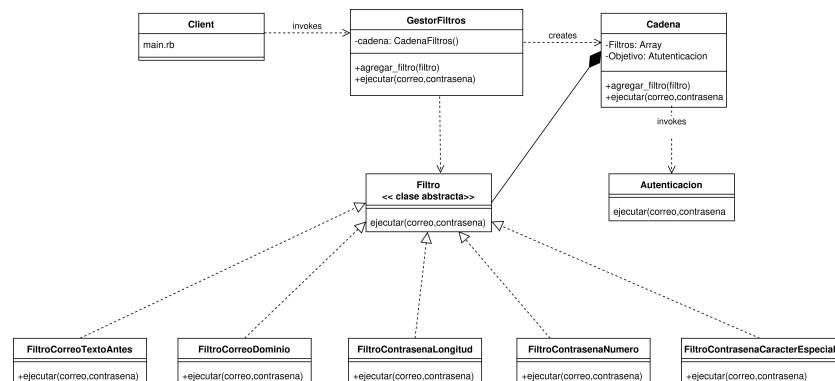
4.2 Patrón utilizado

Para realizar este ejercicio hemos implementado el patrón que se pide, **Filtros de Intercepción** que nos permite añadir una serie de comprobaciones a una petición antes de que llegue a su destino. Actúan como una serie de reglas de seguridad por las que tiene que pasar la petición para continuar adelante.

Las piezas principales de este ejercicio han sido las siguientes:

- **Filtro:** define la interfaz común que todos los filtros deben tener. En este caso, todos deben tener un método `ejecutar`.
- **Filtros concretos:** cada clase que hereda de `Filtro` y tiene que desarrollar el método común por heredar de `Filtro`.
- **CadenaFiltros:** guarda la lista de filtros y los ejecuta en orden, uno tras otro.
- **GestorFiltros:** Se encarga de coordinar todo el proceso de filtros.
- **Objetivo:** la clase que hace el trabajo final, autentica al usuario y solo seguirá adelante si los credenciales del usuario pasan correctamente los filtros.
- **Cliente:** el programa principal que pide los datos y arranca el proceso.

4.3 Diagrama de clases UML



4.4 Cambios respecto al diseño original

En el patrón original `Filtro` se definió como una **interfaz**, es decir, solamente declara las funciones que las clases que la implementen deberán de escribir su código. Sin embargo en ruby no tenemos la

opción de interfaces como en Java, por tanto hemos optado por usar una **clase abstracta** que tendrá el método **ejecutar** definido pero el único código que tiene es un código de error por si alguien intenta utilizarla directamente sin sobrescribirla, así intentamos simular una interfaz en Java pero en ruby de una forma "artesanal"

4.5 Ejemplo de ejecución

```
⚡ 11:05:02 /home/gerardo/Desktop/Universidad/3ºCURSO/2ºCUAT
>_ ruby main.rb
  Sistema de Autenticación
Introduce tu correo: prueba@gmail.com
Introduce tu contraseña: contrasena123,,
OK - El correo tiene texto antes del @
OK - El dominio 'gmail.com' es correcto y válido
OK - La contraseña tiene 8 o más caracteres
OK - La contraseña contiene al menos un número
OK - La contraseña contiene al menos un carácter especial

  Autenticación completada con éxito para: prueba@gmail.com
```